

Les tuples

Définition

```
>>> tup = 'a', 'b', 'c', 'd', 'e'  
>>> print(tup)  
( 'a', 'b', 'c', 'd', 'e')
```

```
>>> tup = ('a', 'b', 'c', 'd', 'e')
```

- Un tuple est une collection d'éléments séparés par des virgules

```
>>> a = (3, 4, 7)  
>>> type(a)  
<class 'tuple'>
```

Définition

- Sont des listes immuables (inchangeables)
- On ne peut pas modifier un tuple après qu'il a été créé, cad qu'on définit le contenu d'un tuple (les objets qu'il doit contenir) lors de sa création, mais on ne peut pas en ajouter ou en retirer par la suite
- Chaînes / listes / tuples
 - Chaînes et tuples ne sont pas modifiables
 - Listes sont modifiables

une fonction qui renvoie un tuple

```
def test():  
    return 3, 4
```

```
>>> a = test()  
>>> a  
(3, 4)  
>>> b, c = test()  
>>> b  
3  
>>> c  
4
```

Note

Après **return**, il aurait été possible d'utiliser une notation avec des parenthèses pour le tuple, par exemple **return (3, 4)**.

Opérations sur les tuples

Opérateurs sur les tuples sont les mêmes que pour les listes sauf que les tuples ne sont pas modifiables

La valeur d'un élément du tuple est obtenue en utilisant la même syntaxe que pour une liste.

```
>>> a[0]
3
>>> a[1]
4
```

- déterminer la taille d'un tuple à l'aide de **len()**
- le parcourir à l'aide d'une boucle **for**
- utiliser l'instruction **in** pour savoir si un élément donné en fait partie
- opérateurs de concaténation
- opérateurs de multiplication
- pas utiliser avec eux, ni l'instruction **del** ni la méthode **remove()**

```
>>> tu1, tu2 = ("a","b"), ("c","d","e")
>>> tu3 = tu1*4 + tu2
>>> tu3
('a', 'b', 'a', 'b', 'a', 'b', 'a', 'b', 'c', 'd', 'e')
>>> for e in tu3:
...     print(e, end=":")
...
a:b:a:b:a:b:a:b:c:d:e:
```

Comment créer un tuple qui contient un seul élément ?

Si on utilise seulement des parenthèses, on n'obtient pas le résultat escompté.

```
>>> a = (3)
>>> a
3
>>> type(a)
int
```

Pour créer un tuple contenant un seul élément, il faut donc utiliser une syntaxe spécifique qui contient une virgule.

```
>>> b = (3,)
>>> b
(3,)
>>> type(b)
<class 'tuple'>
```

Récupérer l'unique valeur présente dans le tuple

Première approche

```
>>> c = b[0]
>>> c
3
```

Deuxième approche

```
>>> d, = b
>>> d
3
```

La deuxième approche avec une virgule `d, = b` est plus légère que la syntaxe qui utilise des crochets `c = b[0]`.

Il est possible d'utiliser la syntaxe `nom_de_variable, =` aussi avec une liste à un élément.

```
>>> u = [5]
>>> v, = u
>>> v
5
```

Dans quels cas on utilise les tuples :

- Affectation multiple : $a, b = 3, 4$
- Permutation de deux variables : $(a, b) = (3, 4)$ #cette syntaxe passe par des tuples qui ne sont pas déclarés explicitement

Liste vs tuples

- Une liste est une séquence mutable pouvant contenir plusieurs autres objets
- On peut insérer des éléments dans une liste à l'aide des méthodes :
 - **append**, **insert** et **extends**
- On peut supprimer des éléments d'une liste grâce au mot-clé **del** ou à la méthode **remove**
- Un tuple est une séquence pouvant contenir des objets
- A la différence de la liste, le tuple ne peut être modifié une fois créé

```
>>> del tu3[2]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object doesn't support item deletion
```